Prinicipal Components Analysis(PCA)

PCA is a dimensionally reduction technique used to reduce the number of features while retaining most of the information in the dataset.

Principal Components

Principal components are new variables created from the original features.

PC1(First Princiapl Component)

- Captures the maximum variance in the data.
- Most important component

PC2(Second Principal Component)

- Captures the remaining variance
- Perpendicular to PC1

PC3,PC4

- Captures progressively less variance

Example

Original features

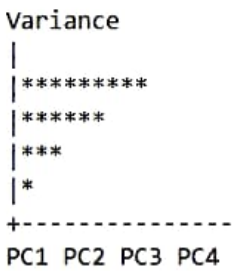
- Height
- Weight
- Age
- Income

Screen Plot

A screen plot shows:

Principal components vs explained variance

Example



Purpose of screen plot

- Find how many components to keep
- Identify the 'elbow point'
- Components after the elbow contribute very little information

When too apply PCA

- Data visualization
- Reduce overfitting
- Highly correlated feature
- Faster training needed

After PCA

- PC1
- PC2

Expalined Variance Ratio

It tells how much information each principal component retains

Example

Component	Variance Explained
PC1	70%
PC2	20%
PC3	7%
PC4	3%

Total = 100%

Interpretation

- PC1 contains 70% of the infromation
- PC2 contains 30% of the infromation
- PC1+PC2 contains 90% of the infromation

So we can keep only:

PC1+PC2

and remove the remaining components

PCA workflow

```
 Data
  |
Standardization
  |
PCA
  |
Principal components
  |
Train model
```

[1]:
```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import pandas as pd

# Load dataset
iris = load_iris()
X = iris.data

# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Explained Variance Ratio
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

```python
# PCA DataFrame
df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

print(df.head())

# Scatter Plot
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")
plt.show()
```

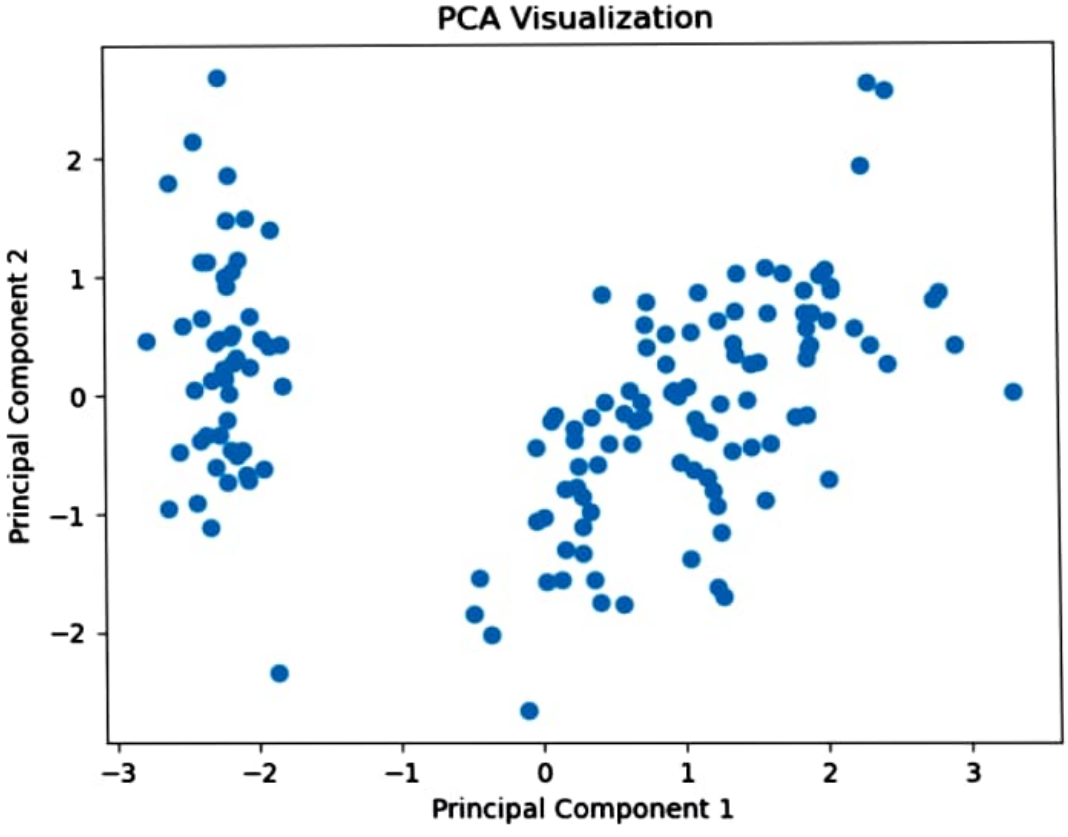
```
Explained Variance Ratio:
[0.72962445 0.22850762]
        PC1       PC2
0 -2.264703  0.480027
1 -2.080961 -0.674134
2 -2.364229 -0.341908
3 -2.299384 -0.597395
4 -2.389842  0.646835
```

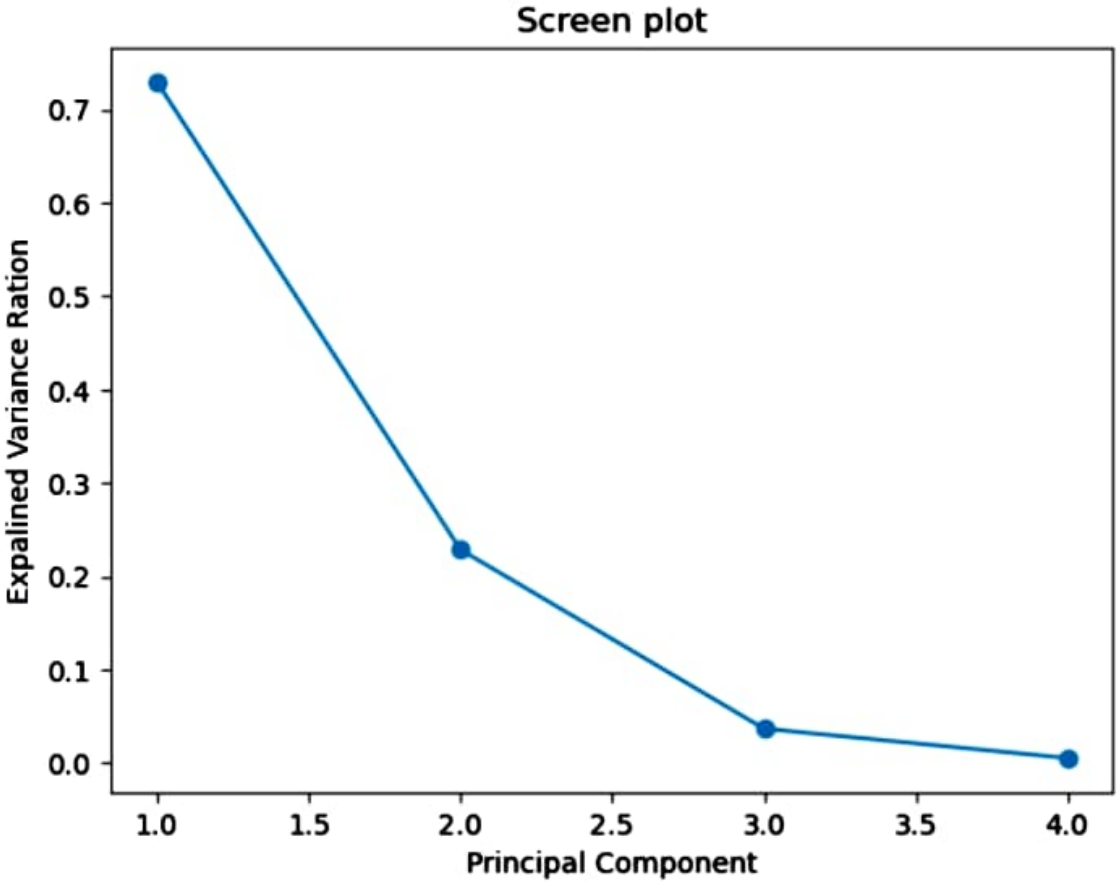
```python
#Screen plot
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca=PCA()
pca.fit(X_scaled)

plt.plot(
    range(1,len(pca.explained_variance_ratio_) +1),
    pca.explained_variance_ratio_,
    marker='o'
)

plt.xlabel("Principal Component")
plt.ylabel("Expalined Variance Ration")
plt.title("Screen plot")
plt.show()
```


PCA Visualization
Principal Component 2
Principal Component 1
2
1
0
−1
−2
−3
−2
−1
0
1
2
3


Screen plot
Expalined Variance Ration
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Principal Component

Cross Validation

It is a technique used to evaluate a ML model by training and testing it on different subsets of the data.

Its main purpose is to check how well the model generalizes to unseen data.

Why is a Single Train-Test split insufficient?

Suppose we split the data as:

- Training Data = 80%
- Testing Data = 20%

Problem

Performance depends on how the data was split.

- A lucky split may give very high accuracy
- An unlucky split may give low accuracy

Results can be misleading

Example

Split 1 Accuracy = 95%

K-Fold Cross Validation

In K-Fold Validation ,the dataset is divided into k equal parts(folds)

Example

```
k=5
Fold1
Fold2
```

Example

```
k=5
Fold1
Fold2
Fold3
Fold4
Fold5
```

Process

Iteration 1

```
Test  → Fold1
Train → Fold2 + Fold3 + Fold4 + Fold5
```

Iteration 2

```
Test  → Fold2
Train → Remaining folds
```

Continue until every fold is used once as test data.

Final Score

Average of all K accuracies

Example ¶

```
Fold1 = 92%
Fold2 = 90%
Fold3 = 94%
Fold4 = 91%
```

Stratified K-Fold Solution

Stratified K-Fold preserves

Example

Original dataset:

```
90% Pass
10%  Fail
```

Each fold:

```
90% Pass
10%  Fail
```

The same class ratio is maintained in every fold.

Benefits of Stratified K-Fold ¶

- More reliable evaluation
- Better for classification problems
- Handles imbalanced datasets
- Maintains class proportions

Example

```
Fold1 = 92%
Fold2 = 90%
Fold3 = 94%
Fold4 = 91%
Fold5 = 93%

(92+90+94+91+93)/5
= 92%
```

Benefits of K-Fold Cross Validation

- Uses all data for training and testing
- More reliable evaluation
- Reduces bias from a single train-test split
- Better estimate of model performance

Stratified K-Fold

Normal K-Fold may create folds with unequal class distributions.

Example dataset

```
90 Pass
10 Fail
```

Some folds may contain very few "Fail" samples.

This can lead to unrelaible evaluation.

K-Fold vs Stratified K-Fold

K-Fold	Stratified K-Fold
Random split into K folds	Preserves class distribution
Suitable for balanced data	Suitable for imbalanced data
May create uneven classes	Keeps class proportions same
General-purpose	Mostly used in classification

```python
#K-Fold Cluster
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold, cross_val_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Model
model = DecisionTreeClassifier()

# K-Fold
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

```
scores = cross_val_score(
    model,
    X,
    y,
    cv=kf
)
print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

```
Scores: [1.         1.         0.93333333 0.93333333 0.93333333]
Mean Accuracy: 0.9600000000000002
```

[6]:
```
#stratiedKFold
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Model
model = DecisionTreeClassifier()

# Stratified K-Fold
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scores = cross_val_score(
    model,
    X,
```

```
scores = cross_val_score(
    model,
    X,
    y,
    cv=skf
)

print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

```
Scores: [1.         0.96666667 0.93333333 0.96666667 0.9        ]
Mean Accuracy: 0.9533333333333335
```

Hyperparameter Tuning

It is the process of finding the best hyperparameter values for a ML model to improve performance.

What are hyperparameters?

Hyperparameters are settings that are chosen before training the model.

Example

Algorithm	Hyperparameters
KNN	n_neighbors (K)
Decision Tree	max_depth, min_samples_split
Random Forest	n_estimators, max_depth
SVM	C, kernel, gamma

Example

For KNN:

```
K = 3
K = 5
K = 7
```

The value of k is hyperparameter.

Why hyperparameter tuning?

Different hyperparameter values can produce different results.

Example

```
K = 1 → Accuracy = 85%
K = 3 → Accuracy = 90%
K = 5 → Accuracy = 95%
```

The model performance changes when the hyperparameter changes.

Goal

Find the best combination of hyperparameter that gives the highset model performance.

Benefits

- Improves model accuracy.
- Reduce overfitting .
- Improves generalization.
- Helps find the optimal model configuration .

Hyperparameter Tuning Methods

1. GridSearchCV

It tries all possible combinations of hyperparameters and selects the combination with the best performance.

Example

Suppose:

```
max_depth = [3,5,7]
min_samples_split = [2,4]
```

GridSearchCV test

All combinations are evaluated using Cross Validation.

Advantages

- Finds the best combination from the given grid
- Easy to understand
- Easy to implement

Disadvantages

- Slow when many parameters exists
- Computationally expensive

2. RandomizedSearchCV

It randomly selects a fixed number of parameter combinations instead of testing all combinations.

Example

Suppose there are:

```
100 possible combinations
```

RandomizedSearchCV may test:

```
10 random combinations
```

instead of all 100.

Advantages

- Faster then gridsearchCV
- Works well with large parameter spaces
- Lower computational cost

Disadvantages

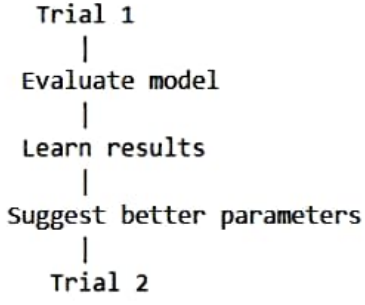
- Many miss the absolute best combination

3. Optuna Basics

Optuna is a modern hyperparameter optimization library.

Instead of checking all combinations or random combinations,it learns from previous trials and intelligently chooses better hyperparameters.

How optuna works



This process continues until the best parameters are found.

Advantages

- Faster optimization
- Smarter search stratergy
- Can stop poor performing trials early(Pruning)
- Widely used in ML and DL

Disadavantages ¶

- Requies installing an additional library
- Slightly more complex than GridSearchCV

Comparison

Method	Search Type	Speed
GridSearchCV	Tests all combinations	Slow
RandomizedSearchCV	Tests random combinations	Faster
Optuna	Learns and chooses better combinations	Fastest

```
!pip install optuna
```

```
Requirement already satisfied: optuna in c:\users\dell-\anaconda3\lib\site-packages (4.9.0)
Requirement already satisfied: alembic>=1.5.0 in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (1.13.3)
Requirement already satisfied: colorlog in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (6.10.1)
Requirement already satisfied: numpy in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (24.1)
Requirement already satisfied: sqlalchemy>=1.4.2 in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (2.0.34)
Requirement already satisfied: tqdm in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (4.66.5)
Requirement already satisfied: PyYAML in c:\users\dell-\anaconda3\lib\site-packages (from optuna) (6.0.1)
Requirement already satisfied: Mako in c:\users\dell-\anaconda3\lib\site-packages (from alembic>=1.5.0->optuna) (1.2.3)
Requirement already satisfied: typing-extensions>=4 in c:\users\dell-\anaconda3\lib\site-packages (from alembic>=1.5.0->optuna) (4.11.0)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\dell-\anaconda3\lib\site-packages (from sqlalchemy>=1.4.2->optuna) (3.0.1)
Requirement already satisfied: colorama in c:\users\dell-\anaconda3\lib\site-packages (from colorlog->optuna) (0.4.6)
Requirement already satisfied: MarkupSafe>=0.9.2 in c:\users\dell-\anaconda3\lib\site-packages (from Mako->alembic>=1.5.0->optuna) (2.1.3)
```

```python
##GridSearchCV code
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

#Dataset
iris=load_iris()
X=iris.data
y=iris.target

#Model
model=DecisionTreeClassifier()

#Parameter grid
param_grid={
    'max_depth':[2,3,4,5],
    'min_samples_split':[2,4,6]}

#Grid search
grid = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring='accuracy'
)
grid.fit(X,y)
print("Best parameters:",grid.best_params_)
print("Best Score:",grid.best_score_)
```

```
Best parameters: {'max_depth': 3, 'min_samples_split': 2}
Best Score: 0.9733333333333334
```

```python
##RandomizedSearchCV code
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

#Dataset
iris=load_iris()
X=iris.data
y=iris.target

#Model
model=DecisionTreeClassifier()

#Parameter distribution
param_dist={
    'max_depth':[2,3,4,5,6,7,8],
    'min_samples_split':[2,4,6,8,10]}

#random search
random_search =RandomizedSearchCV(
    model,
    param_distributions=param_dist,
    n_iter=6,
    cv=5,
    random_state=42
)
random_search.fit(X,y)
print("Best parameters:",random_search.best_params_)
print("Best Score:",random_search.best_score_)
```

```
Best parameters: {'min_samples_split': 4, 'max_depth': 7}
Best Score: 0.9666666666666668
```

```
    score=cross_val_score(
        model,
        X,
        y,
        cv=5
    ).mean()
    return score

#Study
study=optuna.create_study(
    direction="maximize"
)

study.optimize(
    objective,
    n_trials=20
)

print("Best parameters:")
print(study.best_params)

print("Best score:")
print(study.best_value)
```

```python
import optuna
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

#datasets
iris=load_iris()
X=iris.data
y=iris.target

#objective function
def objective(trial):
    max_depth=trial.suggest_int(
        "max_depth",
        2,
        10
    )

    min_samples_split=trial.suggest_int(
        "min_samples_split",
        2,
        10
    )

    model=DecisionTreeClassifier(
        max_depth=max_depth,
        min_samples_split=min_samples_split
    )

    score=cross_val_score(
        model,
        X,
        y,
```

```
[I 2026-06-12 12:35:18,650] A new study created in memory with name: no-name-9f885c6d-9340-43fc-b8de-013f8fafddf5
[I 2026-06-12 12:35:18,684] Trial 0 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_split': 2}. Best is trial 0 wit
h value: 0.9333333333333332.
[I 2026-06-12 12:35:18,729] Trial 1 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_split': 4}. Best is trial 0 wit
h value: 0.9333333333333332.
[I 2026-06-12 12:35:18,764] Trial 2 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_split': 9}. Best is trial 2 wit
h value: 0.9666666666666668.
[I 2026-06-12 12:35:18,801] Trial 3 finished with value: 0.9533333333333334 and parameters: {'max_depth': 4, 'min_samples_split': 4}. Best is trial 2 wit
h value: 0.9666666666666668.
[I 2026-06-12 12:35:18,841] Trial 4 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_split': 7}. Best is trial 2 wit
h value: 0.9666666666666668.
[I 2026-06-12 12:35:18,878] Trial 5 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_split': 3}. Best is trial 2 wit
h value: 0.9666666666666668.
[I 2026-06-12 12:35:18,910] Trial 6 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 8}. Best is trial 2 wit
h value: 0.9666666666666668.
[I 2026-06-12 12:35:18,946] Trial 7 finished with value: 0.9733333333333334 and parameters: {'max_depth': 3, 'min_samples_split': 4}. Best is trial 7 wit
h value: 0.9733333333333334.
[I 2026-06-12 12:35:18,984] Trial 8 finished with value: 0.9666666666666668 and parameters: {'max_depth': 8, 'min_samples_split': 6}. Best is trial 7 wit
h value: 0.9733333333333334.
[I 2026-06-12 12:35:19,025] Trial 9 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 4}. Best is trial 7 wit
h value: 0.9733333333333334.
[I 2026-06-12 12:35:19,079] Trial 10 finished with value: 0.9666666666666668 and parameters: {'max_depth': 10, 'min_samples_split': 10}. Best is trial 7
with value: 0.9733333333333334.
[I 2026-06-12 12:35:19,130] Trial 11 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 10}. Best is trial 7 w
ith value: 0.9733333333333334.
[I 2026-06-12 12:35:19,180] Trial 12 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 6}. Best is trial 7 with value: 0.97
33333333333334.
[I 2026-06-12 12:35:19,230] Trial 13 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 8}. Best is trial 7 wi
th value: 0.9733333333333334.
[I 2026-06-12 12:35:19,277] Trial 14 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 9}. Best is trial 7 with value: 0.97
33333333333334.
[I 2026-06-12 12:35:19,324] Trial 15 finished with value: 0.9666666666666668 and parameters: {'max_depth': 8, 'min_samples_split': 6}. Best is trial 7 wi
th value: 0.9733333333333334.
[I 2026-06-12 12:35:19,372] Trial 16 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 2}. Best is trial 7 with value: 0.97
33333333333334.
[I 2026-06-12 12:35:19,418] Trial 17 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 5}. Best is trial 7 wi
```

```
[I 2026-06-12 12:35:19,025] Trial 9 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 4}. Best is trial 7 wit
h value: 0.9733333333333334.
[I 2026-06-12 12:35:19,079] Trial 10 finished with value: 0.9666666666666668 and parameters: {'max_depth': 10, 'min_samples_split': 10}. Best is trial 7
with value: 0.9733333333333334.
[I 2026-06-12 12:35:19,130] Trial 11 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 10}. Best is trial 7 w
ith value: 0.9733333333333334.
[I 2026-06-12 12:35:19,180] Trial 12 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 6}. Best is trial 7 with value: 0.97
33333333333334.
[I 2026-06-12 12:35:19,230] Trial 13 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 8}. Best is trial 7 wi
th value: 0.9733333333333334.
[I 2026-06-12 12:35:19,277] Trial 14 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 9}. Best is trial 7 with value: 0.97
33333333333334.
[I 2026-06-12 12:35:19,324] Trial 15 finished with value: 0.9666666666666668 and parameters: {'max_depth': 8, 'min_samples_split': 6}. Best is trial 7 wi
th value: 0.9733333333333334.
[I 2026-06-12 12:35:19,372] Trial 16 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 2}. Best is trial 7 with value: 0.97
33333333333334.
[I 2026-06-12 12:35:19,418] Trial 17 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 5}. Best is trial 7 wi
th value: 0.9733333333333334.
[I 2026-06-12 12:35:19,464] Trial 18 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_split': 8}. Best is trial 7 wi
th value: 0.9733333333333334.
[I 2026-06-12 12:35:19,514] Trial 19 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 9}. Best is trial 7 wi
th value: 0.9733333333333334.
Best parameters:
{'max_depth': 3, 'min_samples_split': 4}
Best score:
0.9733333333333334
```